GoSmart

Movilidad inteligente para Riohacha, Maicao y la Troncal del Caribe (La Guajira)

Documento de sustentación del proyecto

Contenido

1. Generalidades y explicación del proyecto
2. Arquitectura y funcionamiento
3. Estado real: qué está hecho y qué no
4. Glosario de términos, herramientas y metodologías
5. Preguntas probables de los docentes, con respuestas sugeridas

Octubre de 2026 · Basado en el repositorio LaszloSM/gosmart

Índice

1. Generalidades del proyecto . . . 3

¿Qué es GoSmart? . . . 3

El problema . . . 3

Usuarios . . . 3

Módulos del MVP . . . 4

Diferenciación . . . 4

Principios del producto . . . 4

2. Arquitectura y funcionamiento . . . 5

Vista general . . . 5

Estructura del código . . . 5

Cómo funciona un viaje, paso a paso . . . 6

El asistente de IA . . . 6

Seguridad y datos . . . 6

Funcionamiento sin señal (offline) . . . 7

3. Estado real del proyecto . . . 8

Fases del plan . . . 8

Cómo se verificó . . . 8

Lo que NO se ha verificado . . . 8

Fuera del alcance por decisión . . . 9

4. Glosario de términos, herramientas y metodologías . . . 10

Lenguajes y frameworks . . . 10

Backend y servicios . . . 10

Conceptos técnicos clave . . . 11

Metodologías y prácticas . . . 12

5. Preguntas probables de los docentes . . . 13

Sobre el problema y el producto . . . 13

Sobre la arquitectura y la tecnología . . . 13

Sobre seguridad, datos y concurrencia . . . 14

Sobre metodología y calidad . . . 14

Sobre limitaciones y futuro . . . 15

1. Generalidades del proyecto

¿Qué es GoSmart?

GoSmart es una aplicación móvil Android (hecha en Flutter) con backend en Supabase que ayuda a moverse en Riohacha, Maicao y por la Troncal del Caribe, el corredor de unos 75 km que une las dos ciudades en La Guajira, Colombia. Convierte información que hoy está dispersa (bloqueos, retenes, inundaciones, tarifas informales, salidas de colectivos) en una decisión de viaje concreta: **por dónde ir, en qué, cuánto pagar y cuándo salir**.

La visión de largo plazo es una **tarjeta universal de transporte** para el país. El MVP (producto mínimo viable) se enfocó primero en el problema más urgente del corredor: la falta de información confiable.

El problema

Un diagnóstico de 17 problemáticas (propuesta "Guajira Movilidad") mostró que el problema central del corredor es la **información**:

- No se sabe si la Troncal está abierta antes de salir (bloqueos, retenes, inundaciones).
- Los modos de transporte más usados son informales (mototaxi, colectivo, TransPadilla) y no tienen tarifas publicadas.
- No hay forma de saber cuándo sale el próximo colectivo ni de reservar un cupo.
- Un viaje real mezcla varios modos (a pie, mototaxi, colectivo, intermunicipal) y nadie lo organiza puerta a puerta.
- Hay preocupación por la seguridad del pasajero.

Usuarios

Usuario	Qué necesita
Viajero del corredor	Va entre Riohacha y Maicao en colectivo, intermunicipal o carro. Necesita saber si la vía está abierta, cuánto cuesta y cuándo sale el próximo cupo.
Usuario urbano	Se mueve dentro de la ciudad en mototaxi, taxi o TransPadilla. Necesita ir de cualquier punto a cualquier punto (dirección, referencia como "frente al Éxito" o un punto en el mapa) y saber cuánto debería pagar.
Conductor	Mototaxista, taxista o conductor de colectivo. Recibe solicitudes, acepta o contraoferta, cobra y cierra el viaje con una mano.
Administrador	Aprueba conductores, modera reportes, mira métricas y apaga funciones si algo falla en el piloto.

Módulos del MVP

Módulo	Qué hace
M1 · Estado del corredor	Reportes ciudadanos de la vía (bloqueo, retén, inundación, hueco, accidente) con confirmación cruzada y reputación. Se comparte por WhatsApp con un link que abre sin instalar la app.
M2 · IA + planificador	Planificador puerta a puerta que corre en el celular (algoritmo de Dijkstra sobre un grafo de transporte) y un asistente de IA que entiende lenguaje natural y usa el planificador como herramienta.
M3 · Viaje encadenado + seguridad	Un viaje con varios tramos y modos, despacho estilo InDrive (el pasajero propone tarifa, el conductor acepta o contraoferta), cupos de colectivo, botón SOS y compartir viaje.
M4 · Pago	Billetera, pago por tramo con saldo, QR del conductor o efectivo confirmado, y un solo recibo por viaje. Recargas con Wompi en modo de pruebas (sandbox).
M5 · Confianza + admin	Calificaciones en las dos direcciones, bloqueo por impagos, reputación del reportante y panel web de administración.

Diferenciación

Waze ya tiene reportes de vía, pero **no** tiene colectivos, TransPadilla, tarifas informales, seguridad del pasajero, el patrón histórico de bloqueos del corredor ni la difusión por WhatsApp. Ninguna app vecina junta en un solo lugar el estado de la Troncal reportado por la gente, los modos informales con su tarifa justa y el viaje encadenado con pago y recibo único.

Principios del producto

- **Antes de salir, ya sé cómo está la vía:** el estado del corredor se ve en un segundo.
- **Honestidad sobre los datos:** si no hay dato se dice "sin datos" con la hora; nunca un estado inventado ni un precio inventado.
- **Funciona en la calle real:** una mano, sol directo, señal débil (contraste 7:1, botones de 48 dp, 72 dp en modo conductor).
- **Lo local primero:** modos, nombres y precios de La Guajira.
- **La cuenta se pide al actuar, no para mirar** (modo invitado).
- **Ética:** los reportes son de tipo cerrado, sin texto libre, para que nunca se señale a personas o comunidades.

2. Arquitectura y funcionamiento

Vista general

La app es **una sola aplicación Flutter** con tres modos según el rol del usuario: pasajero, conductor y administrador (el panel admin se compila también como web). Toda la lógica sensible vive en el servidor.

Capa	Tecnología	Responsabilidad
Cliente móvil	Flutter / Dart, Riverpod, GoRouter	Interfaz, planificador local, cola offline, mapa
Base de datos	Supabase (PostgreSQL)	Datos, seguridad por fila (RLS), funciones RPC atómicas
Autenticación	Supabase Auth	Correo/contraseña y OTP por SMS
Funciones de servidor	Supabase Edge Functions (Deno/TypeScript)	ai-chat (Groq), maps (Google), wompi, push (FCM), delete-account
Mapas	Google Maps (Places, Routes); OSM como respaldo	Mapa, búsqueda de direcciones, rutas calle a calle
IA	Groq (modelos Llama) con function calling	Asistente conversacional
Pagos	Wompi (sandbox)	Recargas de billetera
Web pública	GitHub Pages	Páginas para WhatsApp: estado de la vía y viaje compartido
Operación	GitHub Actions, Sentry	CI, despliegue, respaldos, errores

Estructura del código

Carpeta	Contenido
lib/core	Utilidades: variables de entorno, formato de dinero, región, telemetría, cola offline, links de seguridad
lib/features	Pantallas por funcionalidad: home, trip, driver, wallet, payments, safety, admin, ai_chat, auth…
lib/models	Clases de datos puras (sin Flutter)
lib/planner	Planificador en Dart puro: grafo, tarifas, Dijkstra, conversión a ruta
lib/providers	Estado de la app con Riverpod
lib/services	Clientes de Supabase, Google, IA, búsqueda de lugares, seguridad
lib/widgets	Componentes reutilizables del sistema de diseño (prefijo GS)
supabase/migrations	20 migraciones SQL numeradas (001–020), re-ejecutables

Carpeta	Contenido
backend/functions	Edge Functions en TypeScript
test/ y supabase/tests	Pruebas de Dart (modelos, planificador, widgets) y pruebas SQL

Cómo funciona un viaje, paso a paso

- **1.** El usuario abre la app (sin cuenta) y ve arriba la **línea del corredor**: abierta, con precaución, cerrada o sin datos.
- **2.** Escribe "¿A dónde vas?". La búsqueda va en capas: primero el **directorio propio** de lugares locales, luego Google Places y, si nada sirve, un pin en el mapa.
- **3.** El **planificador** en el celular busca la mejor ruta en el grafo de transporte: engancha el origen y el destino a los nodos más cercanos, evita tramos cerrados y estima un rango de tarifa.
- **4.** El usuario crea un **viaje encadenado** (por ejemplo: mototaxi al terminal → colectivo por la Troncal → a pie). Aquí se le pide la cuenta.
- **5.** Para el mototaxi hace una **solicitud estilo InDrive**; conductores cercanos aceptan o contraofertan. La aceptación es atómica en la base de datos: gana uno solo.
- **6.** Para el colectivo **reserva un cupo** de una salida publicada por un conductor.
- **7.** Cada tramo se **paga al cerrarse** (saldo, QR del conductor o efectivo confirmado). Al final hay **un solo recibo** con el desglose.
- **8.** Durante el viaje tiene **SOS** (123, WhatsApp, SMS) y puede **compartir el viaje** con un link que expira. Al final califica al conductor y viceversa.

El asistente de IA

El celular envía la pregunta a la Edge Function `ai-chat`. Esta llama a Groq con un conjunto de herramientas. Si el modelo pide `plan_trip`, el servidor no inventa la ruta: devuelve la llamada al celular, que ejecuta el planificador local y devuelve el resultado; luego la IA redacta la respuesta. Así, **las rutas siempre vienen del planificador, nunca del modelo**, y la llave de Groq nunca está en el APK. Si Groq falla, la app avisa y el planificador manual sigue funcionando (degradación D8).

Seguridad y datos

- **RLS** en todas las tablas: cada usuario solo ve sus filas.
- Ningún saldo se escribe desde el cliente: todo movimiento de dinero pasa por funciones RPC del servidor con **llave de idempotencia** (evita cobros dobles).
- Nadie puede subirse de rol solo; el estado del conductor lo decide únicamente el administrador.
- Los documentos de conductores van a un bucket privado.
- Las llaves de Google, Groq y Wompi viven en el servidor (secrets), no en el APK (salvo la de Maps SDK, restringida al paquete).
- Política de datos según la **Ley 1581 de 2012** (Habeas Data), en borrador.

Funcionamiento sin señal (offline)

El grafo de transporte y el estado del corredor se guardan en el teléfono, así que el planificador funciona sin internet. Las acciones que escriben (reportes, SOS, etc.) entran a una **cola de sincronización** con su llave de idempotencia y se envían al volver la señal. Si el servidor rechaza una acción, gana el servidor y se avisa al usuario.

3. Estado real del proyecto

Importante para la sustentación: hay que decir la verdad sobre qué está probado y qué no. Los docentes valoran más la honestidad que una demo exagerada.

Fases del plan

Fase	Contenido	Estado
0	Limpieza: migraciones unificadas, una sola billetera, IA fuera del APK, Stripe retirado	Hecha
1	Trabajo de campo: lugares, tarifas, rutas de TransPadilla, km de la Troncal	**No hecha** (requiere campo)
2	Rediseño total de la interfaz (día y noche)	Hecha (rediseñada otra vez el 15-oct)
3	Roles, mapa Google, offline, SOS, compartir viaje, legal	Hecha; push sin probar en vivo
4	Estado del corredor + WhatsApp	Hecha
5	Planificador puerta a puerta + IA con herramientas	Hecha; sin voz
6	Conductor + viaje encadenado	Hecha; sin replaneo automático
7	Pago por tramo + recibo único	Hecha en sandbox
8	Confianza + panel admin	Hecha
9	Piloto en la calle	Preparado, no ejecutado

Cómo se verificó

- **flutter analyze** sin avisos y **pruebas automáticas de Dart** en verde (32 archivos de prueba: modelos, planificador, widgets).
- **Pruebas SQL** de las migraciones en un PostgreSQL 16 dentro de CI, cada migración aplicada dos veces.
- **Prueba de carrera real**: dos conexiones aceptando la misma solicitud a la vez; siempre gana una sola.
- Pruebas de las Edge Functions con Deno; el APK y la versión web compilan en CI.

Lo que NO se ha verificado

- La app en un teléfono real de gama media (gestos del mapa, cámara del QR, SOS).
- El proyecto real de Supabase con todas las migraciones aplicadas de punta a punta.
- Conversaciones reales con Groq, el sandbox real de Wompi y notificaciones push reales (falta proyecto de Firebase).
- Los datos: lugares, tramos, grafo y tarifas son una **semilla aproximada** marcada como no verificada.

Fuera del alcance por decisión

Qué	Por qué
NFC y Tap to Pay	Exige certificación con adquirentes. Se dejó la tabla cards para futuros validadores de transporte público.
Pago único con retención / dinero real en custodia	Retener dinero de terceros exige una figura vigilada por la Superintendencia Financiera.
Wayuunaiki	Debe hacerse con la comunidad wayuu, no con traducción automática.
Modo SMS para conductores sin smartphone	Requiere una pasarela SMS de pago.
Otros municipios	Alcance del MVP.

4. Glosario de términos, herramientas y metodologías

Lenguajes y frameworks

Término	Qué es y cómo se usa en GoSmart
Flutter	Framework de Google para crear apps móviles, web y de escritorio con un solo código. Se usa para toda la app.
Dart	Lenguaje de programación de Flutter. El planificador está escrito en Dart puro.
SQL / PostgreSQL	Lenguaje y motor de base de datos relacional. Toda la lógica crítica (pagos, despacho) está en funciones SQL.
TypeScript / Deno	Lenguaje (JavaScript con tipos) y entorno de ejecución de las Edge Functions.
Riverpod	Librería de manejo de estado en Flutter: los "providers" guardan datos y avisan a las pantallas cuando cambian.
GoRouter	Librería de navegación entre pantallas por rutas (/home, /wallet…), con redirecciones según la sesión.

Backend y servicios

Término	Qué es y cómo se usa en GoSmart
Supabase	Plataforma "backend como servicio" de código abierto: base PostgreSQL, autenticación, almacenamiento, tiempo real y funciones. Se usa en su plan gratuito.
BaaS	Backend as a Service: el servidor lo administra un proveedor y el equipo solo define datos y reglas.
Edge Function	Función pequeña que corre en el servidor de Supabase. Guarda llaves secretas y llama servicios externos.
RPC	Remote Procedure Call: la app llama a una función de la base de datos (ej. accept_ride) en lugar de escribir tablas directamente.
RLS	Row-Level Security: reglas de PostgreSQL que filtran cada fila según el usuario autenticado (auth.uid()).
Migración	Archivo SQL numerado que cambia el esquema de la base de forma ordenada y repetible.
Webhook	Aviso automático por HTTP que un servicio manda a otro cuando pasa algo (Wompi avisa un pago; la base avisa un nuevo reporte para enviar push).
FCM	Firebase Cloud Messaging: servicio de Google para notificaciones push.
Groq	Proveedor de inferencia de IA muy rápida; ejecuta modelos abiertos Llama.

Término	Qué es y cómo se usa en GoSmart
LLM	Large Language Model: modelo de lenguaje grande que entiende y genera texto.
Function calling	Capacidad del LLM de pedir que se ejecute una herramienta (ej. plan_trip) con argumentos estructurados en vez de inventar la respuesta.
Wompi	Pasarela de pagos colombiana (de Bancolombia). Se usa en sandbox, el entorno de pruebas sin dinero real.
Google Places / Routes	APIs de Google para buscar lugares y calcular rutas calle a calle.
OSM / flutter_map	OpenStreetMap: mapa libre; respaldo cuando no hay llave de Google.
Sentry	Servicio de reporte de errores en producción (opcional).
Ley 1581 de 2012	Ley colombiana de protección de datos personales (Habeas Data).

Conceptos técnicos clave

Término	Qué es y cómo se usa en GoSmart
MVP	Producto Mínimo Viable: la versión más pequeña que resuelve el problema principal y permite aprender de usuarios reales.
Grafo	Estructura de nodos (paraderos, terminales) y aristas (trayectos con modo, minutos y tarifa).
Dijkstra	Algoritmo clásico que encuentra el camino de menor costo en un grafo. El costo combina tiempo, espera, tarifa y penalizaciones (noche, bloqueos).
Haversine	Fórmula para calcular la distancia entre dos coordenadas sobre la Tierra.
Gazetteer / directorio	Lista de lugares con nombre popular, alias y coordenadas ("frente al Éxito"). Se busca con trigramas (pg_trgm) sin importar tildes.
Atomicidad	Una operación ocurre completa o no ocurre. Con SELECT … FOR UPDATE se bloquea la fila para que dos conductores no acepten el mismo viaje.
Condición de carrera	Error cuando dos acciones simultáneas compiten por el mismo recurso. Se evita con bloqueo de fila y se prueba con un test de carrera real.
Idempotencia	Repetir una operación da el mismo resultado que hacerla una vez. Cada pago o reporte lleva una llave única para evitar duplicados al reintentar.
Offline-first	Diseñar para que la app funcione sin señal y sincronice después.
Máquina de estados	Conjunto de estados permitidos y transiciones válidas de un viaje (planeado → en curso → completado / cancelado), controlado en SQL.
Degradación con aviso	Si un servicio externo falla, la app sigue funcionando con menos capacidades y lo dice (decisión D8).
Feature flag	Interruptor (app_flags) que enciende o apaga funciones como el despacho o Wompi sin publicar otra versión.

Término	Qué es y cómo se usa en GoSmart
Modo invitado	Usar la app sin cuenta; se pide solo al realizar acciones.

Metodologías y prácticas

Término	Qué es y cómo se usa en GoSmart
Desarrollo por fases	El plan se dividió en 10 fases (0–9) con entregables, estado y verificación por fase.
Registro de decisiones	Cada decisión importante tiene un código (D1–D8, E1–E6, DS1–DS8) con su porqué, para poder defenderla.
Revisiones de plan	Revisión estratégica ("CEO"), de ingeniería y de diseño antes de construir.
CI/CD	Integración y despliegue continuo con GitHub Actions: en cada cambio se analiza, se prueba, se compila el APK y se despliega el backend.
Pruebas automatizadas	Unitarias (modelos, planificador), de widgets (pantallas) y SQL; los 3 casos de referencia del planificador corren en las pruebas.
Diseño centrado en el usuario	Personas definidas, recorrido del usuario (storyboard) y tabla de estados por pantalla (cargando, vacío, error, sin señal).
Sistema de diseño	Tokens (colores, espacios, radios) y componentes reutilizables GS*, documentados en DESIGN.md.
Accesibilidad (WCAG)	Contraste 7:1, áreas táctiles de 48 dp, texto hasta 200 %, el estado nunca depende solo del color.
Desarrollo asistido por IA	Se usó Claude Code como asistente de programación y skills de diseño (impeccable, taste-skill, Emil Kowalski) para revisar calidad visual y animaciones.
Git y GitHub	Control de versiones con ramas y pull requests.

5. Preguntas probables de los docentes

Respuestas sugeridas, cortas y honestas. Adáptenlas con sus palabras.

Sobre el problema y el producto

P: ¿Qué problema resuelve GoSmart y por qué es importante?

La falta de información en el corredor Riohacha–Maicao: no se sabe si la vía está abierta, cuánto cuesta el transporte informal ni cuándo sale un colectivo. Eso causa tiempo perdido, sobrecostos y riesgo.

P: ¿En qué se diferencia de Waze, Google Maps o InDrive?

Waze tiene reportes pero no colectivos, tarifas informales ni seguridad. InDrive tiene despacho pero no estado de la vía ni viajes con varios modos. GoSmart junta todo para el contexto local y se difunde por WhatsApp.

P: ¿Por qué solo Riohacha, Maicao y la Troncal?

Para un MVP conviene un alcance acotado que se pueda validar en campo. La estructura (tablas de lugares, tramos, grafo) permite agregar municipios cargando datos.

P: ¿Cómo validaron que el problema existe?

Con un diagnóstico de 17 problemáticas. Falta la validación con usuarios reales: está preparada como piloto (Fase 9) con 10–20 conductores.

P: ¿Cuál es el modelo de negocio?

Respuesta posible: comisión por viaje o por recarga, convenios con alcaldías (métricas de movilidad) y la visión de tarjeta universal. Aún no está validado; sean honestos con esto.

Sobre la arquitectura y la tecnología

P: ¿Por qué Flutter y no nativo (Kotlin)?

Un solo código para Android, web (panel admin) y a futuro iOS; buen rendimiento y desarrollo rápido para un equipo pequeño.

P: ¿Por qué Supabase y no Firebase o un servidor propio?

Es PostgreSQL real (relacional, SQL, transacciones), tiene RLS, auth y funciones, es de código abierto y su plan gratuito alcanza para el piloto. Las transacciones con bloqueo de fila eran necesarias para pagos y despacho.

P: ¿Qué pasa si Supabase gratis se pausa o se cae?

Se pausa tras 7 días sin uso; se mitiga con un ping programado y un respaldo semanal en GitHub Actions. Es un riesgo conocido, documentado.

P: ¿Cómo funciona el planificador de rutas?

Es un grafo de nodos y trayectos. Con Dijkstra se busca el camino de menor costo (tiempo + espera + tarifa + penalizaciones). Corre en el celular, así funciona sin señal, y evita tramos cerrados.

P: ¿Por qué el planificador corre en el celular y no en el servidor?

Para que funcione offline y responda en milisegundos, y para tener un solo planificador (decisión D6). El grafo pesa menos de 1 MB.

P: ¿La IA puede inventar rutas o precios?

No para las rutas: usa function calling y la ruta la calcula el planificador. Las tarifas se muestran como rango de referencia, y si no hay regla se dice "precio en la ruta".

P: ¿Dónde están las llaves secretas?

En los secrets del servidor (Edge Functions y GitHub). El APK solo lleva la llave anónima de Supabase y la de Maps SDK restringida al paquete.

P: ¿Cómo funciona sin internet?

Grafo y estado del corredor cacheados en el teléfono; las escrituras van a una cola con llave de idempotencia y se envían al volver la señal.

Sobre seguridad, datos y concurrencia

P: ¿Qué pasa si dos conductores aceptan el mismo viaje al mismo tiempo?

La aceptación es una función SQL atómica con SELECT … FOR UPDATE: solo uno gana y el otro recibe "already_taken". Hay una prueba con dos conexiones reales en CI.

P: ¿Cómo evitan cobrar dos veces?

Cada pago lleva una llave de idempotencia y la tabla de pagos tiene restricción única: si se reintenta, no se cobra otra vez.

P: ¿Cómo evitan reportes falsos en la vía?

Tipos cerrados sin texto libre, límite de 6 reportes por hora, confirmación cruzada ponderada por reputación, expiración, y el estado "cerrada" exige confianza ≥ 0,9. El admin puede quitar un reporte.

P: ¿Cómo protegen los datos personales?

RLS en todas las tablas, documentos en bucket privado, links de viaje con expiración, política según Ley 1581 (en borrador) y opción de eliminar cuenta.

P: ¿Es real el dinero de la billetera?

No. Es de prueba (sandbox). Manejar dinero de terceros exige una entidad vigilada por la Superfinanciera; por eso se eligió pago por tramo y recibo único.

Sobre metodología y calidad

P: ¿Qué metodología usaron?

Desarrollo incremental por fases con registro de decisiones, revisiones de plan (estrategia, ingeniería y diseño), CI con pruebas automáticas y verificación al cierre de cada fase.

P: ¿Cómo probaron el sistema?

Pruebas unitarias y de widgets en Dart, pruebas SQL en PostgreSQL, prueba de carrera real, pruebas de Edge Functions con Deno y compilación del APK en CI.

P: ¿Lo probaron en un teléfono real con usuarios?

Todavía no. Respuesta honesta: compila y pasa pruebas, pero falta el piloto en la calle; está la lista de preparación en PILOTO.md.

P: ¿Usaron inteligencia artificial para programar?

Sí, como asistente (Claude Code). Las decisiones, el alcance y la validación son del equipo; todo pasa por pruebas y revisión. Conviene decirlo abiertamente.

P: ¿Cómo aseguraron la accesibilidad?

Contraste 7:1 para leer al sol, botones de 48 dp (72 en modo conductor), texto hasta 200 % y estados con ícono + palabra, no solo color.

Sobre limitaciones y futuro

P: ¿Cuál es la mayor limitación actual?

Los datos: lugares, tarifas y grafo son una semilla aproximada. La Fase 1 de trabajo de campo es la más importante pendiente.

P: ¿Qué harían con más tiempo?

Levantamiento de datos en campo, piloto con conductores, push en vivo, tiempo real en lugar de consultas cada 4–5 s, replaneo automático y edición de rutas desde el panel.

P: ¿Por qué no usaron NFC si la visión es una tarjeta?

NFC entre celulares y Tap to Pay exigen certificación con adquirentes. Se dejó la base (tabla cards) para validadores de transporte público.

P: ¿Escala si crece mucho?

El plan gratuito aguanta un piloto (unas 200 conexiones en tiempo real). Para crecer: plan pago de Supabase, límite de peticiones en las páginas públicas y Realtime.

P: ¿Cómo se incluye a la comunidad wayuu?

No se usa iconografía wayuu como decoración y los reportes nunca señalan comunidades. El wayuunaiki debe construirse con traductores de la comunidad.

Consejos para la presentación

- Muestren primero el problema (la vía cerrada sin aviso) y luego la solución en un solo flujo.
- Tengan a mano los 3 casos de referencia del planificador: centro de Riohacha → Maicao; barrio → Universidad de La Guajira; Maicao → Terminal de Riohacha con la Troncal bloqueada (usa el desvío).
- Si algo no está probado, díganlo antes de que lo pregunten. Eso da confianza.
- Expliquen cada decisión con su porqué (D5, D6, D7, D8): es lo que más valoran los docentes.